# Ders 2 Genişletilmiş Notu: Çevik Yaklaşımlarda Kalite, McCall Kalite Modeli, Kalite Maliyeti ve Gözden Geçirme

## Ders 2 Genişletilmiş Notu: Çevik Yaklaşımlarda Kalite, McCall Kalite Modeli, Kalite Maliyeti ve Gözden Geçirme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Dersin İçeriğine Kısa Bir Bakış
- 2. Yazılım Mühendisliğinde Kalitenin Yeri
- 3. Geleneksel Süreç Modellerinin Hatırlatılması
- 4. Çevik Yaklaşımlar ve Kalite
- 5. Test Güdümlü Geliştirme (TDD)
- 6. Kaliteli Yazılım ve McCall Kalite Üçgeni
- 7. Kalite Faktörleri ile Ölçütler Arasındaki İlişki
- 8. Kalite Sağlama Aktiviteleri
- 9. Kalite Maliyeti
- 10. Yazılım Standartları ve Kalite Yönetim Sistemi
- 11. Gözden Geçirme ve Onaylama
- 12. Gözden Geçirme Tipleri ve Seçenek Matrisi
- 13. Hangi Aşamada Ne Gözden Geçirilir?
- Örnek Sorular


### 1. Dersin İçeriğine Kısa Bir Bakış

Hoca derse, geçen hafta derste olmayanlar için dersin içeriğini hızlıca tekrar ederek başladı. Bağlantı sorunları yüzünden slaytlar bir süre görünmedi. Bu yüzden içerik iki kez, kısmen sözlü olarak anlatıldı.

Dersin ana çizgisi şöyledir. Önce yazılım kalite yaklaşımı ve bunun yazılım mühendisliğindeki yeri ele alınır. Yazılım geliştirme süreç modelleri ve çevik yaklaşımlar kalite açısından hatırlatılır. Ardından yazılım kalite etmenlerinin sınıflandırılması ve McCall'un kalite modeli gelir. Sonra kaliteyi sağlamak için yapılan aktiviteler işlenir: kalite standartları, gözden geçirme ve onaylama (review), inceleme (inspection). İncelemede Fagan'ın inceleme süreci üzerinde durulacaktır. Doğrulama ve geçerleme (verification ve validation) de yazılım kalitesinin önemli kavramlarındandır.

Daha sonra yazılım test sürecine geçilecektir. Hoca, geliştirme sırasında ne kadar dikkat edilirse edilsin hataların bir kısmının ancak testle bulunabildiğini söyledi. Test sürecinde sırayla şunlar yapılır:

- **Birim testi:** Kodlama aşamasında her alt programın, yani algoritmanın test edilmesi.
- **Bütünlük testi (integration testing):** Alt programlar tek tek test edildikten sonra bir bütün olarak doğru çalışıp çalışmadığının doğrulanması. Bu aşamadan sonra hata çıkarsa testlerin tekrarlanması gerekir.

- **Sistem testi:** Bütünlük testinden geçen yazılımın sistem üzerinde doğru çalışıp çalışmadığının test edilmesi.
- **Onaylama testi (kullanıcı onay testi):** Kullanıcının bakış açısıyla, istediği yazılımı elde edip etmediğinin test edilmesi. Bu test, gereksinim analizi ve sistem analizinin karşılığıdır. Kullanıcının isteklerinin yerine getirilip getirilmediğine bakılır.

Test sürecinin adımlarından sonra test teknikleri gelecektir: saydam kutu (white box), kara kutu (black box) ve gri kutu (grey box) testleri, hata giderme (debugging) ve hata düzeyleri. Yazılım ölçümü (software metrics) de kalite için önemli bir konudur. Halstead, McCabe ve McCall gibi ölçütler bir arada ele alınacaktır. Bunlara bağlı olarak yazılım güvenirliği (reliability) ve güvenirlik modelleri işlenecektir.

Test planlama ve yönetimi bölümünde test çalışma ekibinin profesyonel hayatta nasıl kurulduğu ve test senaryolarının nasıl raporlandığı anlatılacaktır. Teslim sonrasında kaliteyi sağlamak için yazılım konfigürasyon yönetimi (software configuration management) ve bakım (maintenance) işlenecektir. Hoca konfigürasyonu "düzen" olarak açıkladı: yazılımın elemanlarının (kod, belgeler, planlar) nasıl bir yapıda olduğunun belirlenmesi ve bu yapının korunması için yapılan işlemlerdir. Bakımda hangi türlerin olduğuna bakılacaktır. Son bölüm kalite iyileştirmedir: CMMI, CMMI yeterlilik düzeyleri, anahtar süreç alanları ve People CMM.

## 2. Yazılım Mühendisliğinde Kalitenin Yeri

Hoca geçen hafta verdiği yazılım mühendisliği tanımını tekrarladı. Yazılım mühendisliği, kaliteli yazılımı en ekonomik şekilde elde etmek için mühendislik çözüm yöntemlerini kullanan bilim dalıdır. Bu tanımda üç temel unsur vardır:

- **Kaliteli yazılım:** Amaç yazılımı herhangi bir şekilde üretmek değil, bu derste sayılacak kalite özelliklerine sahip yazılımı elde etmektir. Kaliteli yazılımı tanımlamak ve elde etmek için kalite yönetimine ihtiyaç vardır.
- **En ekonomik:** Bu, "en ucuz" demek değildir. En iyi yazılımı en uygun maliyetle elde etmek demektir. Bunun için proje yönetimine ihtiyaç vardır.
- **Mühendislik yöntemleri:** Doğrudan kod ya da algoritma yazmak yerine önce analiz, sonra tasarım yapılır ve yazılım bu tasarıma göre geliştirilir. Bitmiş yazılımlar üzerinde ölçüm yapılır. Yazılımın büyüklüğü ve karmaşıklığı ölçülür. Bu veri, bir sonraki yazılımda maliyet tahmini için kullanılır.

  **Hocanın vurgusu:** Yazılım mühendisliği en ucuz yazılımı değil, en iyi yazılımı en uygun maliyetle elde etmeyi hedefler.

Yazılım mühendisliği bu yüzden katmanlı bir yapıdadır. En altta kalite bakış açısı vardır. Onun üstünde süreç (analiz, tasarım, gerçekleştirme, test adımlarından oluşan formal geliştirme süreci), onun üstünde yöntemler, en üstte de bu yöntemleri destekleyen araçlar (tools) bulunur.

Yazılım geliştirme sürecinde esas adımlar yazılımı hazırlamak için yapılan temel işlemlerdir. Bunların yanında destek süreç adımları vardır: geçerleme (validation), doğrulama (verification), formal teknik gözden geçirme, yazılım konfigürasyon yönetimi, yeniden kullanılabilirlik (reusability), yazılım ölçümü (software metrics) ve risk yönetimi.

## 3. Geleneksel Süreç Modellerinin Hatırlatılması

Geleneksel yazılım geliştirme süreç modelleri şelale (waterfall) modeliyle başlar. Hoca bunları şöyle saydı: şelale, artımlı (incremental), spiral, RAD ve birleşik süreç (Unified Process). Birleşik süreç modeli, nesneye yönelik yaklaşımı en iyi yürütmemizi sağlayan modeldir.

### 3.1 RAD Modeli

RAD (Rapid Application Development), adından da anlaşılacağı gibi hızlı uygulama geliştirmedir. Yazılım modüllere bölünür. Her modülü ayrı bir grup hazırlar. Sonra geliştirilen modüller birleştirilir ve bir test işlemi yapılır. RAD modelinin kullanılabilmesi için iki şart vardır:

- Geniş bir ekip olanağı olmalıdır.
- Yazılım modüllere ayrılabilme özelliği göstermelidir.

### 3.2 Spiral Model

Spiral (sarmal) model aynı zamanda evrimsel bir yazılım geliştirme modelidir. Her sarmal şu aşamalardan oluşur: müşteriyle iletişim, planlama, risk analizi, mühendislik, yapım (construction) ve teslim, müşteri değerlendirmesi. Bu aşamaların hepsi her çevrimde uygulanır.

Hoca slayttaki şekil üzerinden çevrimleri anlattı. En içteki çevrimde bir prototip geliştirilir. Sonraki çevrimde yeni ürün geliştirilir ve kullanıma alınır. Daha sonraki çevrimlerde ürüne yeni eklemeler yapılır, ürün zenginleştirilir. Her seferinde aynı aşamalar baştan uygulanır.

## 4. Çevik Yaklaşımlar ve Kalite

Hoca çevik yöntemlerin ayrıntılı tanımının Yazılım Mühendisliği dersinde yapıldığını hatırlattı. Orada Scrum'un ve Kanban'ın özellikleri ve XP'nin çevik yaklaşımların ilk ortaya çıkış noktası olduğu anlatılmıştı. Bu derste yalnızca kalite açısından bir hatırlatma yapıldı.

### 4.1 Çevik Yaklaşımların Ortaya Çıkışı

Çevik yaklaşımlar geleneksel yöntemlerden sonra kullanılmaya başlandı. Hocaya göre XP (Extreme Programming) ile başladılar. 1999 sonu, 2000 başı gibi bir manifestoyla, bir "başkaldırıyla" ortaya çıktılar. Amaç, artan müşteri taleplerini daha hızlı karşılamak ve geleneksel modellere bir alternatif sunmaktı. Çevik modeller, yazılım geliştirme sırasında gereksinimlerin geç ortaya çıkmasını önlemek için kullanılır. Geçen 20 yılda sektörde yaygınlaştılar.

> **Ders sonrası not:** Hoca manifestonun 1999 sonu ile 2000 başında çıktığını söyledi. XP büyük olasılıkla 1990'ların sonunda ortaya çıkmıştır. Çevik Manifesto (Agile Manifesto) ise büyük olasılıkla 2001'de yayımlanmıştır.

Peki diğer süreç modelleri kayboldu mu? Hayır. Hata kritik bir yazılımda spiral model hâlâ kullanılır, çünkü hatayı evrimsel olarak en aza indirmeyi sağlar. Var olan geleneksel bir yazılımı yenilemek istediğimizde de şelale modeli kullanılır.

> **Ders sonrası not:** Hoca bir cümlede askeri yazılımlar gibi hata kritik yazılımlarda "çevik yaklaşımlar kullanılıyor" dedi, hemen ardından da bu tür yazılımlarda spiral modelin kullanıldığını söyledi. Bağlama göre kastedilen, hata kritik yazılımlarda geleneksel modellerin hâlâ kullanıldığıdır.

### 4.2 XP'nin Değerleri ve Uygulamaları

XP dört değer çerçevesinde oluşturuldu: basitlik, iletişim, geri besleme ve cesaret. Hoca bunları şöyle açıkladı: daha basit çözümlerle başla, iletişimi ön planda tut, ekipten geri besleme al ve yeni yöntemleri daha hızlı kullanıma almaktan çekinme.

XP'yi diğer çevik yaklaşımlardan ayıran uygulamalar şunlardır:

- Eşli programlama (pair programming).
- Sürekli yeniden tasarım (refactoring).
- Ortak kod mülkiyeti.
- Sürekli bütünleştirme (continuous integration).

### 4.3 Çevik Modellerde Kalite Neden Farklı?

Hoca çevik modellerle geleneksel modeller arasındaki kalite farklarını üç noktada topladı.

Birincisi, çevik modellerde dinamik kalite değerlendirme statik kalite değerlendirmeden daha çok kullanılır. Dinamik kalite değerlendirme testtir. Test yapabilmek için yazılımı çalıştırmak gerekir. Statik kalite değerlendirmeye örnek ise incelemedir (inspection). İncelemede yazılım çalıştırılmadan değerlendirilir.

İkincisi, çevik modellerde kalite ile ilgili işlemler sürecin daha erken adımlarında başlar. Kalite aktiviteleri daha kısa aralıklarla, daha sık yapılır.

Üçüncüsü, çevik yaklaşımlarda kullanıcı hikayeleri (user stories), birim testler ve müşteri geri beslemesi hem doğruluğu hem de sürekli bir kalite değerlendirmesini sağlar.

## 5. Test Güdümlü Geliştirme (TDD)

Çevik yaklaşımlarda test güdümlü geliştirme (Test Driven Development, TDD) kullanılır. TDD, küçük ve hızlı iterasyonlarla, yazılım geliştirme araçlarıyla önceden hazırlanmış testler yazılarak ilerleyen bir geliştirme yöntemidir. Kısaltmasıyla çok karşılaşılır.

TDD'de daha sistem analizi yapılırken yazılımın nasıl test edileceği, kullanıcı testinin nasıl yapılacağı düşünülür ve test planları hazırlanır. Yazılımın en küçük parçaları olan birimler için test senaryoları (test case) baştan oluşturulur. Adımlar şöyledir:

- Birim test senaryolarını yaz.
- Testleri çalıştır ve kod henüz yazılmadığı için başarısız olduklarını gör.
- Kod yazma aşamasında birimleri çalıştırarak test et. Bütünlük testini bekleme.
- Tasarım sırasında da kod yazarak test senaryolarının çalıştırılmasını sağla.
- Bütünleştirme ilerledikçe test senaryolarını çalıştır ve testten geçtiklerini gör.
- Hatalar ortaya çıktıkça kodu yeniden düzenle (refactor) ve tekrar kullan.

Hocaya göre bu son adım sürece bir dinamiklik katar.

Hoca TDD ile ilgili bir anı da anlattı. TDD üzerine ilk çalışmaları yapan uzmanlardan Hakan Erdoğmuş, yaklaşık 10 yıl önce bölümün düzenlediği Ulusal Yazılım Mühendisliği Sempozyumu'na davetli konuşmacı olarak gelmişti. O sırada ABD'de IEEE Software dergisinin editörü olarak çalışıyordu. Araştırmasına göre TDD programcı üretkenliğini artırıyordu, fakat kalitede bazı eksiklikler görülmüştü. Test sonuçları ne kadar iyileştirmeyle desteklenirse sonucun o kadar iyileşeceği sonucuna varmıştı. Çevik yaklaşımların çıkışından bu yana 20 yıl, TDD üzerindeki iyileştirme çalışmalarının başlangıcından bu yana da 10 yıl geçti. TDD sektörde yaygın olarak kullanılıyor.

## 6. Kaliteli Yazılım ve McCall Kalite Üçgeni

### 6.1 Kaliteli Yazılım Nedir?

Hoca kalite temel kavramlarına geçerken bu bölümde nelerin işleneceğini söyledi: yazılım kalite faktörleri, McCall modeli, kalite sağlama aktiviteleri, kısaca kalite maliyet modelleri ve doğrulama-geçerlemenin kalitedeki yeri. Asıl ağırlık kalite faktörleri ve kalite sağlama aktiviteleri üzerindedir.

Kaliteli yazılım, önceden belirlenmiş işlevsel gereksinimlere ve yazılım geliştirme standartlarına uygun, kabul edilebilir hata düzeyinde olan yazılımdır. Bu tanımın üç parçası vardır:

- Gereksinim spesifikasyonunda belirtilmiş bütün işleri yapar. İşlevsel olmayan özellikleri de sağlar.
- Standartlara uygun geliştirilmiştir.
- Hata düzeyi kabul edilebilirdir. Hata sayısına sıfır diyemeyiz, ama en aza indirilmiş olmalıdır.

### 6.2 McCall Kalite Üçgeni

Kalite özellikleri McCall kalite üçgeninde üç grupta toplanır. Üçgenin kenarları yazılım ürününe üç farklı bakışı gösterir: ürünün kullanımı, ürünün geçişi (bir ortamdan başka bir ortama aktarılması, başka bir yazılımla birlikte kullanılması) ve üründe değişiklik (bakım).

- **Ürün kullanımı (product operation):** Kullanıcının bakış açısıyla istenen özelliklerdir.
  - Doğruluk (correctness).
  - Güvenirlik (reliability): Doğruluğa yüzde kaç güvenebiliriz?
  - Kullanışlılık (usability).
  - Bütünlük (integrity, security).
  - Etkinlik (efficiency): Performans.
- **Ürün geçişi (product transition):** Ürünün geçişini yapan kişilerin bakış açısıyla önemlidir.
  - Taşınabilirlik (portability).
  - Yeniden kullanılabilirlik (reusability).
  - Birlikte çalışabilirlik (interoperability).

- **Üründe değişiklik (product revision):** Üründe değişiklik yaparken önemli olan özelliklerdir.
  - Bakım yapılabilirlik (maintainability).
  - Esneklik (flexibility).
  - Test edilebilirlik (testability).

  **Ek bilgi (derste anlatılmadı):** McCall modeli büyük olasılıkla 1977'de Jim McCall ve arkadaşlarının ABD hava kuvvetleri için yaptığı bir çalışmada önerilmiştir. Modelin özgün hâlinde, derste sayılan 11 faktör ve bunlara bağlı ölçütler (criteria) bulunur.

Hoca bazı özellikleri ayrıca açtı. Taşınabilirlik, yazılımın bir ortamdan, bir işletim sisteminden başka bir işletim sistemine taşınabilmesidir. Yeniden kullanılabilirlik çok önemlidir. Yazılım mühendisliğinin önemli alt başlıklarından biridir; bazı kaynaklarda ayrı bir kitap ya da bölüm olarak ele alınır. Hazırladığımız bir yazılımın bir parçasının, bütününün ya da yalnızca tasarımının başka bir yerde kullanılmasıdır. Nesneye yönelik yaklaşım yeniden kullanılabilirliği daha çok destekler. Nesneye yönelik yaklaşımda tasarım kalıpları (design patterns) da yeniden kullanımı destekler. Alana özgü modelleme (domain-specific modeling) de yeniden kullanımı destekleyen kavramlardandır.

Birlikte çalışabilirlik, yazılımın başka yazılımlarla uyumlu çalışmasıdır. Başka yazılımlara veri gönderebiliyor mu, onların veri tabanını kullanabiliyor mu, buna bakılır. Hoca yeniden kullanılabilirliğin üründe değişiklik yaparken de önem taşıdığını ekledi. Üründe değişikliği kolaylaştıran alt kavramlar da vardır. Örneğin belgeleme varsa değişiklik yapmak daha kolaydır.

### 6.3 Kalite Özelliklerinin Tanımları

Hoca bir sonraki slaytta özelliklerin yazılı tanımlarını verdi:

| Özellik | Tanım |
|---|---|
| Doğruluk (correctness) | Spesifikasyonlara uygunluk, müşteri isteklerini karşılama derecesi |
| Güvenirlik (reliability) | Tasarlanan işlemleri ne kadar duyarlılıkla yerine getirdiği; doğru sonuca ne kadar güvenilebildiği |
| Verimlilik (efficiency) | İşlevleri yerine getirmek için gereken bilgi işlem kaynaklarının ve kodlama giderinin, gerçekleştirilen işlevlerle uyumlu olması |
| Güvenlik (security) | Yetkisiz kişilerin yazılıma girmesinin engellenmesi |
| Kullanışlılık (usability) | Öğrenme, işletme, girdi hazırlama ve çıktı yorumlama kolaylığı; kullanıcı dostu (user friendly) olma |
| Bakım yapılabilirlik (maintainability) | Hata bulma kolaylığı; hatanın yerini bulup düzeltme olanağı |
| Esneklik (flexibility) | Yazılımda değişiklik yapma kolaylığı |
| Test edilebilirlik (testability) | Sınama yapılabilirlik |
| Taşınabilirlik (portability) | Yazılımın farklı donanımlarda kullanılabilmesi |
| Yeniden kullanılabilirlik (reusability) | Yazılımın tümünün ya da bir bölümünün başka bir uygulamada kullanılabilmesi |
| Birlikte çalışabilirlik (interoperability) | Bir sistemin başka bir sistemle birlikte çalışabilmesi |

Hoca interoperability için Türkçede daha önce "bağlanabilirlik" dendiğini, ama bunun anlamı tam vermediğini söyledi. "Birlikte çalışabilirlik" daha uzun bir isim olsa da daha iyi anlatır. Kullanışlılık için de "kullanım kolaylığı" ya da "kullanıcı dostu" gibi farklı adlar görülebilir.

# 7. Kalite Faktörleri ile Ölçütler Arasındaki İlişki

## 7.1 Faktörler Doğrudan Ölçülemez

McCall'un yazılım kalite modelinde kalite faktörleri doğrudan ölçülemez. Her faktör, bazı ölçülebilir özelliklere bağlı bir doğrusal regresyon denklemiyle kestirilir:

$$F_q = c_1 m_1 + c_2 m_2 + \cdots + c_n m_n$$

Burada $F_q$ bir kalite faktörünü, $m_1, m_2, \ldots, m_n$ yazılım ölçütlerini (metrikleri) gösterir. Ölçütler yazılımı değerlendirebileceğimiz, daha ölçülebilir kavramlardır. $c_i$ katsayıları her ölçütün faktöre katkı oranıdır. Hoca her ölçütün katkı oranının farklı olduğunu söyledi. Ölçütlerin bir kısmı da yine başka şeylere bağlıdır; faktör, bunların bir bileşimidir.

## 7.2 Faktör-Ölçüt Şekli

Hoca Yazılım Mühendisliği dersinde de gösterdiği ilişkilendirme şeklini açtı. Şeklin sol tarafında kalite faktörleri, sağ tarafında ölçülebilir kavramlar (ya da başka kalite faktörleri) vardır. Bir faktör birden çok ölçüte bağlıdır. Hocanın derste okuduğu eşleştirmeler şunlardır:

- **Doğruluk:** izlenebilirlik (traceability), tam olma (completeness), uyumluluk (consistency).
- **Güvenirlik:** doğruluk (accuracy), uyumluluk, hata toleransı (error tolerance). Hata toleransı, hata ortaya çıktığında yazılımın bunu tolere edip çalışmasına devam edebilmesidir.
- **Etkinlik:** çalışma etkinliği (çalışma hızı, cevap süresi), bellek etkinliği (ne kadar bilgi saklayabildiği).
- **Bütünlük:** erişim kontrolü, erişimin denetlenmesi.
- **Kullanılabilirlik:** çalışabilirlik (operability), eğitim, iletişim yapılabilirlik.
- **Bakım yapılabilirlik:** basitlik, "hassasiyet", ifade edilebilirlik, modülerlik. İfade edilebilirlik burada belgelemedir: yazılımın belgelemesi varsa bakım daha kolay yapılır.
- **Test edilebilirlik:** basitlik (basit, anlaşılır bir tasarım var mı?), araç desteği (geliştirirken bir araç kullanılmış mı?), ifade edilebilirlik (test edebilmek için belgeleme var mı?), modülerlik (yazılım modüllere ayrılarak mı hazırlanmış, modül tanımları var mı?).
- **Yeniden kullanılabilirlik:** basitlik, genelleştirilebilirlik (tasarım kalıpları gibi genel bir yapı var mı?), yazılım sistemi bağımsızlığı, modülerlik. Bir modül başka modüllere çok bağımlıysa yeniden kullanılabilirliği düşüktür.
- **Birlikte çalışabilirlik:** modülerlik, iletişim genelliği, veri genelliği. Başka veri tabanlarını kullanabilmesi için veri genelleştirmesi yapılabiliyor mu, bu önem kazanır.

  **Ders sonrası not:** Hoca test edilebilirlikteki araç desteğini (instrumentation) "geliştirirken bir araç kullanılmış mı" diye açıkladı. McCall modelinde bu ölçüt büyük olasılıkla yazılımın kendi çalışmasını izleme ve ölçme olanağı sunmasını, yani kodun içine yerleştirilen izleme ve ölçme noktalarını anlatır.

Hoca bölümü şöyle bağladı: Sağ taraftakiler daha ölçülebilir ya da başka kalite faktörlerine bağlı ölçütlerdir, sol taraftakiler kalite özellikleridir. Bir kalite faktörünü hesaplamak için birden fazla metrik kullanılır ve her birinin katkı oranı farklıdır. Böylece bir faktör hakkında sayısal bir değer bulabiliriz. Kaliteli yazılım elde etmeyi hedefliyorsak hangi özelliklere bakmamız gerektiğini ve bunları nasıl hesaplayacağımızı bu model gösterir.

# 8. Kalite Sağlama Aktiviteleri

## 8.1 Hatayı Erken Bulmak

Kaliteli yazılım için hata ve eksikleri anında bulup düzeltmek gerekir. Bir basamakta yapılan hata sonraki basamakları etkiler ve süreç ilerledikçe hatayı bulma olasılığı azalır. Hoca, Yazılım Mühendisliği dersindeki "hatayı erken bulalım" diyagramlarını hatırlattı.

> **Hocanın vurgusu:** Hatayı ne kadar erken ararsak bulma olasılığımız o kadar artar, harcayacağımız para o kadar azalır.

### 8.2 Aktivitelerin Listesi

Kaliteli yazılım elde etmek için yapılması gereken aktiviteler şunlardır:

- **Yazılım standartlarını kullanmak.**
- **Doğrulama ve geçerleme yapmak.**
- **Gözden geçirme (review):** İlk adımlardan başlayarak yapılan her analiz, her tasarım, müşteriden dinlenen her bilgi gözden geçirilir ve denetlenir.
- **İnceleme (inspection):** Gözden geçirmenin bir üst işidir. Hataları bulmak için kod ve belgeler daha ayrıntılı incelenir.
- **Dinamik değerlendirme:** Test yapmak.
- **Formal bir yazılım geliştirme süreç modeli benimseyip uygulamak.**
- **Hata ve kusur analizi, değişim yönetimi, konfigürasyon yönetimi, kullanıcı eğitimi ve risk yönetimi.**

### 8.3 Doğrulama ve Geçerleme

Doğrulama (verification) ve geçerleme (validation) Türkçede neredeyse aynı anlama gelen kelimelerdir, ama kavramlar farklıdır:

- **Doğrulama (verification):** Yazılımı doğru yolla ürettik mi? Türkçede bu kavramın "doğrulama" kelimesiyle eşleştirilmesine karar verilmiştir.
- **Geçerleme (validation):** İstenen ürünü ürettik mi? Ürün, istediğimiz ürün mü? Bunun için de "geçerleme" kelimesi kararlaştırılmıştır.

  **Hocanın vurgusu:** Doğrulama ve geçerleme Türkçede benzer kelimelerdir ama anlamları farklıdır. Doğrulama sürece, geçerleme ürüne bakar.

### 8.4 Süreç Adımlarında Kalite İçin Dikkat Edilecekler

Hoca esas adımları (gereksinim analizi, tasarım, gerçekleştirme, test) tek tek ele alıp her birinde kalite için neye ağırlık verileceğini söyledi:

- **Gereksinim analizi:** Sistemin kolayca öğrenilip kullanılabilmesine, istenmeyen hallerin planlı olarak değiştirilmesine çaba gösterilir. Sistemi öğrenip analiz ederken, onu nasıl daha kolay kullanıma açacağımız düşünülür.
- **Genel (mimari) tasarım:** Modüller ayrılırken taşınabilirlik ve birlikte çalışabilirliğe önem verilir. Modüller verimli belirlenmeli ve birbirinden bağımsız olmalıdır.
- **Ayrıntılı tasarım ve gerçekleştirme:** Doğruluk faktörüne ağırlık verilir. Her modülün işini doğru yapması için algoritma doğru kurulur.
- **Test:** Doğruluk ve olabildiğince performans gibi faktörlerin ölçülmesine gayret edilir.

## 9. Kalite Maliyeti

### 9.1 Hata Bulma Gideri ve Hata Bulma Olasılığı

Hoca kalite maliyetinden önce sözlü olarak söz etmişti, sonra ilgili diyagramı da gösterdi. Diyagramın yatay ekseni yazılım geliştirme sürecidir. Bir tarafta hata bulma gideri, diğer tarafta hata bulma olasılığı vardır. Süreç ilerledikçe hata bulma gideri artar: daha fazla para ve daha fazla yazılımcı zamanı harcamak gerekir. Aynı zamanda hata bulma olasılığı düşer. Bu yüzden gözden geçirme aşaması, bir süreç modeli kullanmak ve bir standart kullanmak önemlidir.

> **Hocanın vurgusu:** Kaliteyi baştan sağlamanın, yani hatayı önlemenin maliyeti, kaliteyi sonradan sağlamaya çalışmanın (hata bulma ve düzeltme) maliyetinden her zaman azdır. Kaliteyi test aşamasına bırakmamak gerekir.

Kaliteyi baştan sağlama aktiviteleri, az önce sayılan standart kullanma, doğrulama, gözden geçirme ve formal bir süreç modeli kullanmadır. Hoca TDD'de bu iki tarafın iç içe geçtiğini de belirtti: TDD'de test geliştirmenin başında yapıldığı için önleme ile bulma birbirine karışır.

### 9.2 Hata Düzeltmenin Göreli Maliyeti

Hoca, hatanın bulunduğu aşamaya göre düzeltme maliyetinin nasıl katlanarak arttığını göstermek için şu sayıları verdi:

| Hatanın bulunduğu aşama | Göreli düzeltme maliyeti |
|---|---|
| Gereksinim analizi | 0,2 |
| Tasarım | 0,5 |
| Birim testi | 2 |
| Kabul testi (müşteriye teslimde müşteri bulmuşsa) | 5 |
| Bakım (kullanıma alındıktan sonra) | 20 |

Bu sayılara göre bakım aşamasında bulunan bir hata, gereksinim analizinde bulunan bir hatadan $20/0{,}2 = 100$ kat daha pahalıya düzeltilir (ara adım eklendi). Bakım aşamasında hatayı düzeltmek en zorudur, çünkü bunun için tüm yazılımı yeniden anlamak gerekir.

Hoca gözden geçirme konusunda da bir örnek verdi. Yazılım geliştirmedeki hataların %65'i tasarım aşamasında yapılır. Gözden geçirme sırasında bu hataların %75'i bulunup düzeltilir. Gözden geçirmede bulunamayan hatalar daha sonra da düzeltilebilir. Ama test sırasında düzeltilirse maliyet katlanarak artar. Hoca bu sayıları konunun önemini görmek için verdiğini, kaynaklarda benzer sayıların görülebileceğini söyledi.

### 9.3 Hata Önleme Maliyeti

Hata önleme maliyetine girenler şunlardır:

- Kalite planlama: Yazılımı geliştirirken kaliteyi baştan planlamak.
- Formal teknik gözden geçirme ve inceleme (review, inspection).
- TDD kullanılıyorsa baştan itibaren yapılan küçük testler.
- Eğitim: Kullanıcılara eğitim ve yazılım geliştiricilere kullanılan yazılım elemanlarıyla ilgili eğitim.

### 9.4 İç ve Dış Hata Maliyeti

Hata maliyeti ikiye ayrılır:

- **İç hata maliyeti:** Hata, yazılım kullanıma alınmadan önce çıkar. Yazılım geliştirme ekibinin içinde fark edilir ve halledilir. Hata müşteriyle karşılaşmaz. En azından müşterinin güvenini kaybetme sorunu çıkmaz.
- **Dış hata maliyeti:** Hata, kullanımdan sonra kullanıcıya yansır. Müşterinin güveni de kaybedildiği için daha maliyetlidir.

Hoca ayrıca bir kalite maliyet modeli şekli gösterdi. Şekilde geliştirme ve yazılım sağlama birimleri ile yönetimin kalite maliyetleri (kalite kontrol işlemlerinin nasıl yapılacağı) vardı. Bunlar da iç hata ve dış hata maliyeti olarak ayrılıyordu. Hoca bu konunun ekonomi tarafına fazla girdiğini, şekli yalnız bilgi için gösterdiğini söyledi.

## 10. Yazılım Standartları ve Kalite Yönetim Sistemi

### 10.1 Standart Örnekleri

Kalite sağlama aktivitelerinin ilki standart kullanmaktır. Hoca öğrencilerden yazılım standardı örneği istedi (bkz. Örnek Sorular). Öğrenciler ISO, CMMI ve IEEE'yi saydı. Hoca CMMI'yi yazılım geliştirme yeteneklerinin olgunluğunu artırma modeli olarak açıkladı. IEEE'nin uluslararası elektrik-elektronik standartlarını hazırladığını, bunların içinde yazılım geliştirmenin her aşamasına ait standartlar olduğunu söyledi. PMI (Project Management Institute) ise yalnızca yazılım için değil, tüm projelerin yönetimi için standartlar içerir.

### 10.2 Standartların Sınıflandırılması

Yazılım mühendisliği standartları üç şekilde sınıflandırılır:

- **Yazılım geliştirme aşamasına göre:** proje planı (proje yönetimi), gereksinim tanımı (yazılım gereksinim standardı), tasarım spesifikasyonu (design specification), kodlama, konfigürasyon yönetimi ve konfigürasyon planı standartları.
- **Kaynağına göre:** Standardı kim hazırlamış? Uluslararası (ISO, Uluslararası Standartlar Organizasyonu; IEEE), ulusal (örneğin Türk Standartları Enstitüsü) ya da askeri (DoD, Department of Defense, ABD Savunma Bakanlığı kaynaklı standartlar).
- **Değerlendirme ölçü tipine göre.**

Hoca bu arada kalite yönetim aktiviteleri içinde doğrulama ve geçerlemenin de yer aldığını hatırlattı. Burada statik ve dinamik yazılım değerlendirmesinin sağlanması, gözden geçirme tipinin belirlenmesi ve kalite yapısını kimin oluşturacağı belirlenir.

### 10.3 Kalite Yönetim Sistemi

Bir kurumda kalite yönetim sistemi kurulurken genel standartlardan (örneğin ISO 9126, DoD) biri ya da birkaçı temel alınır. Bir kalite el kitabı hazırlanır. Kurumun standartları ve kalite kontrolleri bu kitapta tanımlanır. Kurumda geliştirilen her projede bu standartlar ve kalite kontrolleri temel alınır. Her proje için ayrıca, bu standartlara ek olarak başka standartların kullanılıp kullanılmayacağına karar verilir.

> **Ek bilgi (derste anlatılmadı):** ISO 9126 bir yazılım ürün kalitesi standardıdır. Büyük olasılıkla 2011'den itibaren yerini ISO/IEC 25010 standardına bırakmıştır. ISO/IEC 25010 de McCall modeline benzer biçimde kaliteyi özelliklere ve alt özelliklere ayırır.

Yazılım kalite yönetiminde şunlar belirlenir:

- Hangi standart kullanılacak?
- Hangi geliştirme süreç modeli kullanılacak?
- Hangi ölçüm yöntemi kullanılacak, yani neler ve nasıl ölçülecek?
- Konfigürasyon yönetimi nasıl yapılacak?
- Doğrulama ve geçerleme için neler yapılacak?

Hocaya göre kaliteyi güvence altına almak, standartları kullanmakla eş anlamlıdır. Kalite yönetimi kaliteyi güvence altına alır, yüksek kalitede ürün geliştirmeyi sağlar, ölçümle kalite derecesini belirler ve sonraki projelerin tahmini için temel oluşturur.

### 10.4 Kaliteyi Belirlemek

Geliştirme sürecinde kaliteyi yükseltmek için gereksinim analizinde, tasarımda ve ayrıntılı tasarımda yapılacak işler (Bölüm 8.4) burada yeniden hatırlatıldı. Kaliteyi belirlemek için ise şu yollar kullanılabilir:

- Doğrudan nesnel (objektif) kalite ölçütleri: McCabe ile karmaşıklığı ölçmek, Halstead ile işlem sayısını yorumlamak.
- Dolaylı ölçütler.
- Uzman yazılımcılarla tartışmak.

> **Ek bilgi (derste anlatılmadı):** McCabe'nin çevrimsel karmaşıklığı (cyclomatic complexity), programın kontrol akış grafından hesaplanır ve bağımsız yol sayısını verir. Halstead ölçütleri ise programdaki işleç (operator) ve işlenen (operand) sayılarından program hacmi ve çaba gibi değerler türetir. Her iki ölçüt de ilerideki yazılım ölçümü bölümünde ele alınacaktır.

## 11. Gözden Geçirme ve Onaylama

### 11.1 Gözden Geçirme Nedir?

Hoca aradan sonra, hata önleme işlemlerini hatırlattı: gözden geçirme, standart kullanma ve formal bir süreç modeli kullanma. Standartlar ve süreç modelleri işlenmişti; sıra gözden

geçirme ve onaylamaya geldi.

Yazılım geliştirmede her basamak tamamlanınca, durak noktalarında (milestone) yapılan işler gözden geçirilir ve onaylanır. En temel model olan şelale modelinde bu, her adımın sonunda yapılır: adım tamamlanır, gözden geçirilir, onaylanır. Şelale dışındaki modellerde her dönemin bitmesi beklenmez. Daha sık aralıklarla durak noktaları belirlenir ve buralarda gözden geçirme yapılır.

Gözden geçirme bir eleme işlemidir. Hata ve eksiklerin zamanında bulunup giderilmesini sağlar. Daha sık aralıklarla gözden geçirme yapılırsa hata bulma gideri düşer, hata bulma olasılığı artar. Bölüm 9.2'deki %65 ve %75 örneği bu bağlamda verildi.

### 11.2 Gözden Geçirmenin Amacı

- Yazılımdaki işlevsel ve mantıksal hataları, kodlama dilinin hatalı kullanımını bulmak ve düzeltmek.
- Her adımda yazılımın gereksinimlere uygun olarak geliştirildiğini onaylamak.
- Belirlenen standartlara uygun gidilip gidilmediğini kontrol etmek. Örneğin hazırlanan gereksinim spesifikasyonu belgesi, IEEE'nin gereksinim spesifikasyonu standardına (SRS, Software Requirements Specification) uygun raporlanmış mı? Kod, önceden belirlenen kodlama dili standartlarına uygun mu?
- Proje yönetimini kolaylaştırmak ve müşteriye dağıtılacak ürün belgelerinin daha kaliteli hazırlanmasını sağlamak.

### 11.3 Gözden Geçirme Ne Değildir?

Gözden geçirme (review), teknik uzmanların çalışmalarının teknik uzmanlar tarafından değerlendirildiği bir toplantıdır. Toplantı yerine belgeler üzerinden de yapılabilir: belgeler dağıtılır, incelenir ve gözden geçirme raporları hazırlanır. Kısacası yazılım mühendisliği sürecinde üretilen ürünlerin teknik değerlendirmesidir. Bir kalite sağlama aktivitesidir.

Gözden geçirme aynı zamanda bir eğitim ortamıdır. Hata zamanında söylendiğinde programın kodunu yazan, algoritmayı hazırlayan ya da tasarımı yapan kişi hatasını anlamış olur.

Gözden geçirme şunlar değildir:

- Bir proje özeti değildir. Yazılım geliştiricinin çıkıp yazılımı her noktada nasıl geliştirdiğini uzun uzun anlattığı bir sunum değildir.
- Yazılım geliştirme ekibinin (kişisel ya da politik) değerlendirmesi değildir.
- Yalnızca bilgi verme toplantısı değildir.

Gözden geçirme, teknik olarak hazırlanan işin kurallara göre değerlendirildiği ve hataların net olarak belirtildiği bir toplantı ya da incelemedir. Sonunda raporlar gözden geçirilir ve bir hata raporu sunulur.

### 11.4 Gözden Geçirme Toplantısındaki Roller

Gözden geçirme toplantısında şu kişiler bulunur ve gözden geçirme orada, anında yapılır:

- Gözden geçirme lideri (görevlisi).
- Kullanıcıların temsilcisi.
- Kalite güvence ekibinden, standartlara uygunluğu kontrol eden biri.
- Kayıt yapan kişi (kayıtçı).
- Üretici, yani yazılım geliştirici ekipten biri.

Hepsinin bulunması idealdir. Ama yazılımın büyüklüğüne göre roller azalabilir ya da çoğalabilir. Örneğin küçük bir yazılım evinde ayrı bir kalite güvence ekibi olmayabilir. O zaman belirlenen standartları ekip kendi içinde kontrol eder. Ayrı bir gözden geçirme lideri olmayabilir. Bir kullanıcı temsilcisi, ekibin kendi içinden belirlenen bir kayıtçı ve bir üretici yeterli olabilir.

## 12. Gözden Geçirme Tipleri ve Seçenek Matrisi

Gözden geçirme ve inceleme tipleri şunlardır:

- **Formal olmayan gözden geçirme (informal review):** "Review" dendiğinde esas anlaşılan bu ilk tiptir.
- **Yapısal denetim (walkthrough):** Gözden geçirme ile inceleme arasındaki geçiş hâlidir.
- **İnceleme (inspection).**
- **Round Robin Peer Review:** İncelemenin özel bir hâlidir.

Hoca üç ana tipin farklarını bir seçenek matrisiyle gösterdi (GG: formal olmayan gözden geçirme, WT: walkthrough, İnc.: inspection):

| Özellik | GG | WT | İnc. |
|---|---|---|---|
| Gözden geçirme lideri | yok | var | var |
| Toplantı takvimi | belki | var | var |
| Gözden geçirenlerin ön hazırlığı | belki | var | var |
| Yazılımcı sunum yapar | belki | var | yok |
| Hataları bulmak için kontrol listesi (checklist) | yok | yok | var |
| Bulunan hataların sınıflandırılması | yok | yok | var |
| Kayıtçı ayrıca not alır | belki | var | var |

Matrisin bazı satırlarını hoca ayrıca açıkladı:

- Walkthrough'da yazılımcı sunum yapar, çünkü walkthrough tüm adımların birlikte gözden geçirilmesidir.
- Inspection'da yazılımcı sunum yapmaz. İnceleyenler incelemeyi raporlar üzerinden yapar.
- Inspection'da her aşamadan geçerken "uyuyor mu, şu maddeyi sağlıyor mu" diye bakılan bir kontrol listesi oluşturulur.
- Inspection'da bulunan hatalar sınıflandırılır: şu tip hatalardan şu kadar, şu tip hatalardan şu kadar.
- Formal olmayan gözden geçirme çok kurallı değildir. Toplantı sırasında ürün ya da yarı ürünlere hızlıca bakılır, hata var mı diye kontrol edilir ve hata not alınır. Sınıflandırma ya da ayrı bir kayıtçı bazen vardır, bazen yoktur.

Her projede inspection yapılmaz, ama formal olmayan gözden geçirme çok daha sık yapılır.

Hoca soru olup olmadığını sorduktan sonra küçük projelerdeki duruma değindi. Küçük projelerde düzeltme işlemini doğrudan yazılımı geliştiren kişi de yapabilir. Ancak eksikler üçüncü bir kişi tarafından daha kolay ve daha duyarlı biçimde görülür. Bu yüzden gözden geçirmenin üçüncü bir göz tarafından yapılması daha doğrudur.

> **Hocanın vurgusu:** Gözden geçirmeyi yazılımı geliştiren değil, üçüncü bir göz yapmalıdır. Geliştirici kendi hatasını tekrarlama eğilimindedir, bu yüzden bu yola pek gidilmemelidir.
>
> **Ek bilgi (derste anlatılmadı):** Fagan incelemesi (Fagan inspection), büyük olasılıkla 1970'lerde IBM'de Michael Fagan tarafından geliştirilmiştir. Planlama, genel bakış, hazırlık, inceleme toplantısı, yeniden çalışma ve takip adımlarından oluşan, formal ve rolleri tanımlı bir süreçtir. Derste bir sonraki hafta işleneceği söylendi.

## 13. Hangi Aşamada Ne Gözden Geçirilir?

### 13.1 Gözden Geçirme Noktaları

Gözden geçirme ve onaylama için yazılım geliştirme sürecinin genel durak noktaları (milestone) kullanılır. Hoca milestone kavramını sistem analizinden hatırlatıyordu. Gözden geçirme genelde şu noktalarda yapılır: sistem analizi bittiğinde, yazılım geliştirme planı tamamlandığında, gereksinim analizi tamamlandığında, tasarım ve kodlama tamamlandığında, test tamamlandığında ve bakım-onarım işleminin sonunda. Proje büyükse daha sık aralıklar seçilmelidir.

### 13.2 Sistem Analizi

Sistem analizinde şunlar gözden geçirilir: spesifikasyonların yazılıma ilişkin işlev ve görevler olarak dağıtımının doğruluğu, ana işlevler, ara birimler, performans tanımları, alternatifler arasındaki seçimlerin tutarlılığı, sistemin test edilme esasları ve sistem öğeleri arasındaki uyum.

Hoca bunu analiz aşamasında ne yaptığımız üzerinden açıkladı. Alt sistemler arasındaki ara birimleri anlamaya çalışırız. Sistemde performans ve tasarım kısıtları var mı, bakarız. Sistemi anlayınca problemi çözmek için hangi alternatif çözümleri kullanacağımıza karar veririz ve bu seçimlerde tutarsızlık var mı, bakarız. Teknik olarak yapılabilir (feasible) olmayan bir yapı var mı, değerlendiririz.

Sistemi doğru anladık mı? Bunu kullanıcılarla gözden geçiririz. Kullanıcılara bir bağlam (context) veri akış diyagramı ve birinci düzey veri akış diyagramı çizilir. Yanlarına gidilip "Bunu biz doğru anladık mı, sisteminiz gerçekten böyle mi çalışıyor?" diye sorulur ve onay alınır.

### 13.3 Yazılım Geliştirme Planı

Yazılım geliştirme planında projenin gerçekleşmesi sırasındaki risklere bakılır. Bu amaçla şunlar gözden geçirilir:

- Yazılım sorunu doğru tanımlanmış mı? Terimler doğru belirtilmiş mi?
- Kaynaklar yeterli mi, gerçekten sağlanabilecek mi?
- Görevler uygun mu?
- Gider ve zamanlama tahminleri tutarlı mı?

### 13.4 Gereksinim Analizi

Gereksinim analizinde sistem düzeyindeki tutarlılık ve doğruluk üzerinde durulur:

- Gereksinim spesifikasyonu, standardın bölümlerine uygun mu?
- Analiz tutarlı ve doğru mu? Sorun doğru bölümlere ayrılmış mı?
- Gereksinimler işlevsel ve işlevsel olmayan diye ayrılır. Özellikle işlevsel olmayan gereksinimler, sistem tanımı ve işletim sistemi çerçevesinde yapılabilecek şeyler mi?
- Firma ya da kullanıcı, oluşturulan prototip üzerinde görüş ve onay veriyor mu?
- Gereksinimler zaman, kaynak ve bütçe bakımından gerçekleştirilebilir mi?

### 13.5 Tasarım

Tasarım iki düzeyde gözden geçirilir. Yazılım tasarımı basamağında veri yapıları, program yapısı ve işlemler ele alınır. Önce ön (genel) tasarım, sonra ayrıntılı tasarım gözden geçirilir.

**Ön (genel) tasarım:** Gereksinimler tasarıma doğru dönüştürülmüş mü? Gereksinim analizinde yapılan iş ve alt sistem tanımları, belirtilen kaynaklarla sağlanabilecek şekilde tasarıma yansıtılmış mı? Yazılım mimarisi doğru oluşturulmuş mu? Modüller etkin ve işlevsel olarak bağımsız mı? Program yapısı buna uygun mu?

Klasik tasarımda bir yapı diyagramı (structure chart) çizilir ve modüller hiyerarşik olarak sıralanır. Gözden geçirmede bu modül hiyerarşisinin doğru olup olmadığı, doğru çalışıp çalışmayacağı incelenir. Modül bağımsızlığı sağlanıyor mu, modüller arasındaki geçişler doğru mu, modüllerin birbiriyle ilişkisi ve iç bağımsızlığı uygun mu, bakılır.

Nesneye yönelik yaklaşımda nesne hiyerarşisi ve sınıf diyagramları incelenir. Burada kavramsal sınıf diyagramına değil, tasarım sınıf diyagramına bakılır. Sınıfların gerçekleştirdiği fonksiyonlar ve bu fonksiyonları tetikleyen mesajlar doğru ilişkilendirilmiş mi, etkileşimler takip edilerek kontrol edilir.

Tasarımın gözden geçirilmesini kullanıcı değil, yazılım mühendisleri, yani tasarımı yapan uzmanlar yapmalıdır.

**Ayrıntılı tasarım:** Her modülün kendi iç yapısındaki algoritma tasarımı doğru mu? Algoritma işlevleri doğru yerine getirecek ve doğru sonuç verecek şekilde tasarlanmış mı? Algoritma hatası var mı?

### 13.6 Kodlama ve Test

**Kodlama:** Tasarımda belirtilenler, seçilen programlama diliyle doğru kodlanmış mı? Kod hem programlama diline hem de tasarımdaki algoritmaya uygun mu?

**Test:** Bütün test işlemleri planlanan test planına göre yapılmış mı? Testler hedefine uygun mu?

Hoca bölümü şöyle özetledi: Gözden geçirmenin ne olduğunu, ne olmadığını, yazılım geliştirmenin hangi adımlarında yapıldığını ve gözden geçirme matrisiyle formal olmayan gözden geçirme, walkthrough ve inspection arasındaki farkları gördük. İncelemeye (inspection) bu hafta geçilmedi.

## Örnek Sorular

Bu derste ekran görüntüsü yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

**Soru 1** (derste sözlü, 01:44:18): Yazılım mühendisliği standartlarına örnek verebilir misiniz?

> **Çözüm (derste verildi):** Öğrenciler ISO, CMMI ve IEEE'yi saydı. Hoca bunlara PMI'ı (Project Management Institute) ekledi, ama PMI'ın yalnızca yazılım için değil, tüm projelerin yönetimi için standartlar içerdiğini belirtti. Genel olarak yazılım standartları için ISO ve IEEE sayılabilir. Ayrıca ulusal (Türk Standartları Enstitüsü) ve askeri (DoD) standartlar vardır.

**Soru 2** (derste sözlü, 02:27:03): Sistem analizinde sistemi doğru anlayıp anlamadığımızı kiminle ve nasıl gözden geçiririz?

> **Çözüm (derste verildi):** Kullanıcılarla. Kullanıcılara bağlam (context) veri akış diyagramı ve birinci düzey veri akış diyagramı gösterilir ve "Sisteminiz gerçekten böyle mi çalışıyor?" diye onay alınır.

**Soru 3** (derste sözlü, 02:31:08): Klasik tasarımda ne yapıyorduk ve tasarım gözden geçirmesinde buna nasıl bakılır?

> **Çözüm (derste verildi):** Klasik tasarımda bir yapı diyagramı çizilir ve modüller hiyerarşik olarak sıralanır. Gözden geçirmede modül hiyerarşisinin doğru olup olmadığına, modül bağımsızlığına ve modüller arası geçişlerin doğruluğuna bakılır. Bu gözden geçirmeyi kullanıcı değil, yazılım tasarımcıları yapar.

## Dersin Toparlaması

Bu derste önce yazılım mühendisliğinde kalitenin yeri hatırlatıldı: amaç kaliteli yazılımı en uygun maliyetle elde etmektir. Geleneksel süreç modellerinden RAD ve spiral model, ardından çevik yaklaşımlar ve XP kalite açısından ele alındı. Çevik modellerde dinamik değerlendirmenin (test) öne çıktığı, kalite aktivitelerinin daha erken ve daha sık yapıldığı, TDD'de testlerin koddan önce yazıldığı görüldü. Sonra kaliteli yazılım tanımlandı ve McCall kalite üçgeninin üç grubu (ürün kullanımı, ürün geçişi, üründe değişiklik) ile 11 kalite faktörü işlendi. Faktörlerin doğrudan ölçülemediği, ölçülebilir ölçütlerin ağırlıklı toplamıyla kestirildiği anlatıldı.

Dersin ikinci yarısı kaliteyi sağlama aktivitelerine ayrıldı. Doğrulama ile geçerleme ayrımı yapıldı. Kalite maliyetinde hatayı önlemenin her zaman hatayı sonradan bulup düzeltmekten ucuz olduğu, göreli düzeltme maliyetinin gereksinim analizindeki 0,2'den bakımdaki 20'ye çıktığı gösterildi. Standartların sınıflandırılması ve kalite yönetim sistemi anlatıldı. Son olarak gözden geçirmenin amacı, ne olmadığı, toplantı rolleri, gözden geçirme tiplerinin seçenek matrisi ve her durak noktasında neyin gözden geçirildiği işlendi. Hoca bir sonraki derste inceleme (inspection) ve Fagan'ın inceleme süreciyle, ardından doğrulama ve geçerleme konusuyla devam edileceğini söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.